**DELICIOUSthings**

# MVP - Communication structurée autour des créations personnalisées

Cahier de cadrage produit • User Stories • Use Cases • Planification

Version de travail - 13 août 2026

**Hypothèse centrale : Deliciousthings ne remplace pas WhatsApp ou Instagram. Il transforme les échanges dispersés en un projet clair, partagé et actionnable.**

# 1. Vision du MVP

Deliciousthings est un outil de communication structurée entre une personne qui souhaite faire réaliser une création pâtissière personnalisée et un pâtissier / cake designer capable de la réaliser.

La valeur du produit ne vient pas d'une messagerie supplémentaire. Elle vient de sa capacité à convertir une intention, des références visuelles, des contraintes et des échanges en une représentation commune du projet : le Backprojekt.

> **Promesse MVP : « Transformez une demande floue en projet clair, partagé avec votre pâtissier. »**

## 1.1 Problème à résoudre

- Les demandes arrivent par Instagram, WhatsApp, e-mail ou oralement et sont souvent incomplètes.
- Les informations importantes sont réparties dans plusieurs messages : date, nombre de personnes, budget, style, allergies, livraison, références.
- Le client ne sait pas toujours quelles informations sont nécessaires pour rendre son idée réalisable.
- Le pâtissier doit reformuler, relancer et reconstruire mentalement le projet avant de pouvoir décider ou proposer une adaptation.
- Les décisions prises au fil de la discussion restent difficiles à retrouver et à distinguer des simples idées.

## 1.2 Compétence distinctive

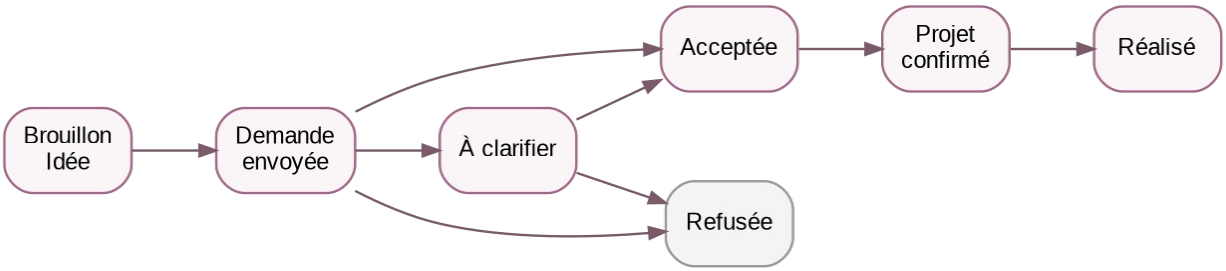
La compétence à développer est la suivante : capter une intention humaine et faire évoluer les échanges jusqu'à obtenir une compréhension commune, structurée et réalisable du projet.

| Canal classique | Deliciousthings |
|---|---|
| Transporte des messages | Organise les messages autour d'un projet |
| Historique chronologique | État actuel + historique des décisions |
| Informations dispersées | Brief partagé et toujours visible |
| Le client doit savoir quoi demander | Le produit guide progressivement la clarification |
| Conversation = résultat | Conversation = matière première du projet |

# 2. Périmètre fonctionnel

## 2.1 Boucle de valeur MVP

1. Le client décrit son idée.
2. Deliciousthings structure les informations essentielles.
3. La demande est envoyée à un artisan précis via sa page / son lien.
4. L'artisan accepte, refuse ou demande des clarifications.
5. Les deux parties échangent dans le contexte du projet.
6. Les décisions importantes sont intégrées au résumé partagé.
7. Le projet passe à un état confirmé ou clôturé.




*Cycle de vie simplifié du Backprojekt*

## 2.2 Fonctionnalités incluses

| Bloc | Fonctionnalités MVP | Priorité |
|---|---|---|
| Compte | Inscription, connexion, rôle/contextes client et artisan | P0 |
| Profil artisan | Nom, bio courte, zone, spécialités, lien public de demande | P0 |
| Demande | Formulaire progressif : occasion, date, portions, budget indicatif, description, références, contraintes | P0 |
| Backprojekt | Résumé partagé, statut, participants, données structurées, journal des échanges | P0 |
| Communication | Fil de messages asynchrone lié au projet, pièces jointes/images limitées | P0 |
| Clarification | Questions de l'artisan + mise à jour des champs du projet | P0 |
| Décisions | Marquer une proposition / information comme validée | P1 |
| Dashboard artisan | Demandes reçues, statut, dernière activité, prochaine action | P0 |
| Dashboard client | Mes projets et leur état | P0 |
| Notifications | E-mail sur nouvelle demande / nouveau message important | P1 |

## 2.3 Explicitement hors MVP

- Marketplace ouverte avec recherche et matching automatique.
- Paiements, acomptes et facturation.
- Chat temps réel, présence, appels audio/vidéo.
- Gestion avancée de production, stock ou comptabilité.
- Calendrier complexe et synchronisation externe.
- Génération IA obligatoire. L'IA peut être expérimentée plus tard comme assistant de clarification.
- Réseau social / feed communautaire généraliste.

# 3. Acteurs et User Stories

## 3.1 Acteurs principaux

| Acteur | Objectif | Douleur principale |
|---|---|---|
| Client | Faire comprendre son idée et obtenir une création adaptée | Ne sait pas toujours quoi préciser et perd le fil des décisions |
| Artisan | Recevoir une demande qualifiée et décider rapidement si elle est réalisable | Informations incomplètes, relances, conversations dispersées |
| Système Deliciousthings | Maintenir une représentation commune du projet | Éviter que le produit devienne seulement une messagerie |

## 3.2 User Stories - Client

| ID | User Story | Prio |
|---|---|---|
| US-C01 | En tant que client, je veux commencer une demande même si mon idée est encore incomplète, afin de ne pas devoir tout connaître dès le départ. | P0 |
| US-C02 | Je veux ajouter une date, un nombre de portions, un budget indicatif, des préférences et des images afin que l'artisan comprenne le contexte. | P0 |
| US-C03 | Je veux envoyer ma demande à un artisan précis depuis son lien public afin d'éviter de reformuler mon besoin ailleurs. | P0 |
| US-C04 | Je veux voir un résumé toujours à jour de mon projet afin de savoir ce qui a été décidé. | P0 |
| US-C05 | Je veux répondre aux questions de l'artisan dans le projet afin que les réponses restent contextualisées. | P0 |
| US-C06 | Je veux savoir quelles informations sont encore manquantes afin de pouvoir faire avancer le projet. | P1 |
| US-C07 | Je veux valider une proposition importante afin de distinguer une décision d'une simple discussion. | P1 |
| US-C08 | Je veux voir le statut du projet afin de savoir si ma demande est reçue, à clarifier, acceptée ou confirmée. | P0 |

## 3.3 User Stories - Artisan

| ID | User Story | Prio |
|---|---|---|
| US-A01 | En tant qu'artisan, je veux disposer d'un lien de demande public afin de le | P0 |

| | | |
|---|---|---|
| | placer dans ma bio Instagram ou l'envoyer à un prospect. | |
| US-A02 | Je veux recevoir des demandes déjà structurées afin de réduire le temps passé à collecter les informations de base. | P0 |
| US-A03 | Je veux voir immédiatement date, portions, budget et références afin d'évaluer rapidement la faisabilité. | P0 |
| US-A04 | Je veux accepter, refuser ou demander des précisions afin de qualifier la demande. | P0 |
| US-A05 | Je veux poser des questions et proposer des adaptations dans le contexte du projet afin de conserver une trace exploitable. | P0 |
| US-A06 | Je veux mettre à jour le résumé du projet afin que les deux parties voient la même version de la demande. | P0 |
| US-A07 | Je veux marquer certains éléments comme décidés afin de sécuriser la compréhension commune. | P1 |
| US-A08 | Je veux voir mes demandes classées par statut et dernière activité afin de savoir quoi traiter ensuite. | P0 |

### 3.4 Critères d'acceptation transversaux

- Un projet possède un résumé lisible sans devoir relire l'intégralité des messages.
- Chaque message est rattaché à un seul projet.
- Le système distingue au minimum les états : brouillon, envoyée, à clarifier, acceptée, refusée, confirmée, réalisée/archivée.
- Une modification importante du brief est visible par les deux participants.
- Un utilisateur ne peut accéder qu'aux projets dont il est participant, sauf page publique explicitement prévue.
- L'expérience reste utilisable sans IA et sans communication temps réel.

## 4. Use-Case Diagrams

Les diagrammes suivants décrivent le périmètre fonctionnel du MVP, sans représenter les détails techniques ou les écrans.

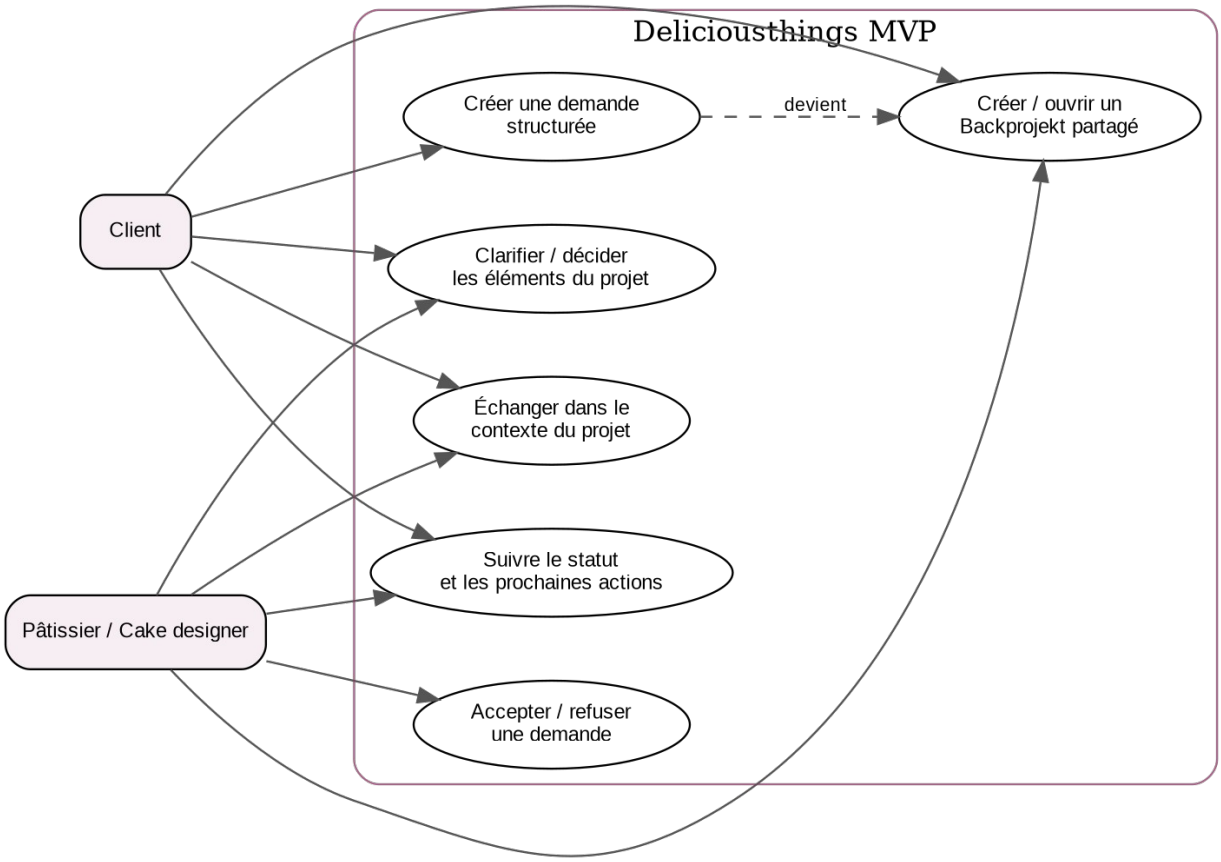



*Diagramme 1 - Cas d'usage globaux*

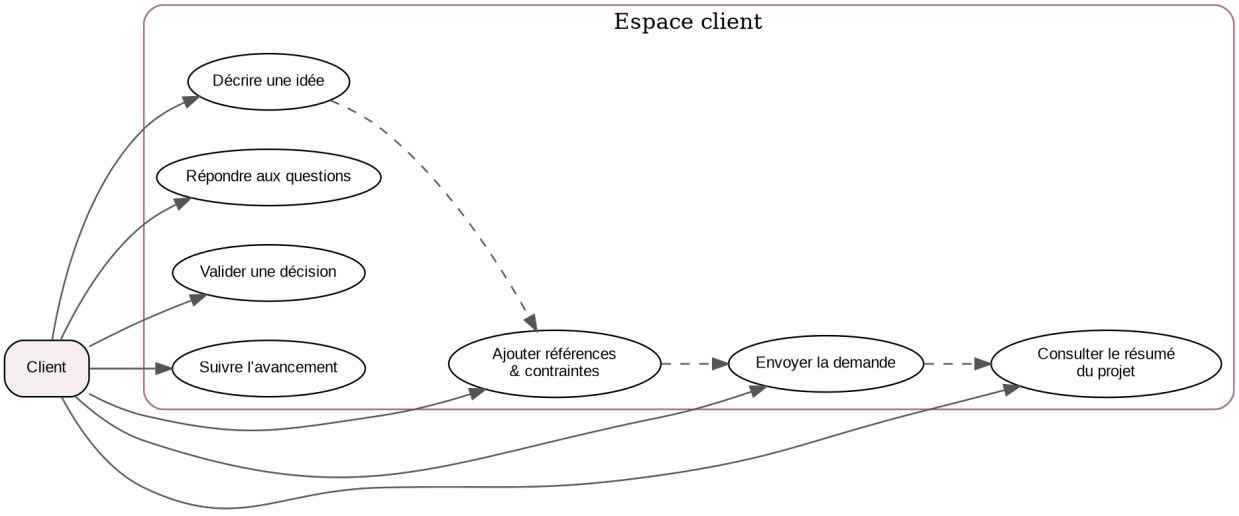



*Diagramme 2 - Cas d'usage du client*

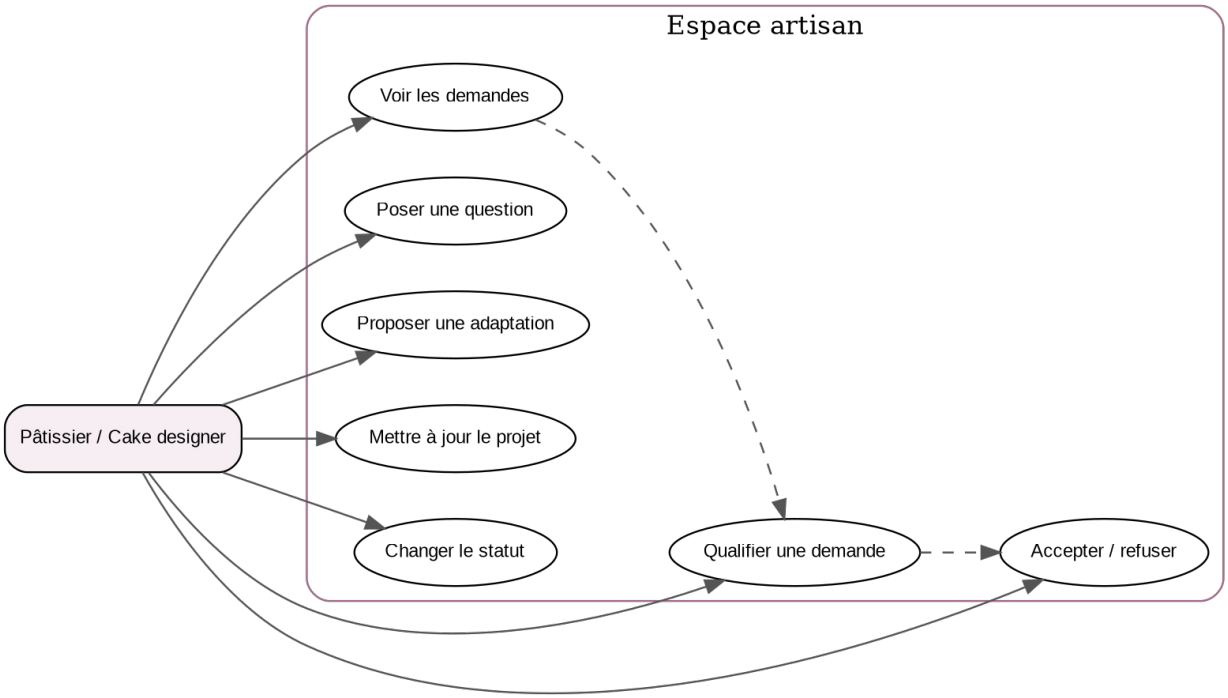



*Diagramme 3 - Cas d'usage de l'artisan*

# 5. Cas d'usage détaillés

| Use Case | Acteurs | Précondition | Résultat | Flux principal |
|---|---|---|---|---|
| UC-01 - Soumettre une demande | Client | Le client ouvre le lien public d'un artisan. | Un Backprojekt est créé et transmis à l'artisan. | Décrire l'idée → renseigner les informations essentielles → ajouter des références → vérifier le résumé → envoyer. |
| UC-02 - Qualifier une demande | Artisan | Une demande envoyée existe. | La demande est acceptée, refusée ou passe à « à clarifier ». | Ouvrir la demande → lire le brief → vérifier contraintes clés → choisir une action → éventuellement poser une question. |
| UC-03 - Clarifier le projet | Client + Artisan | Le projet nécessite une précision. | Le brief partagé est enrichi et l'ambiguïté est réduite. | Question → réponse → mise à jour d'un champ / ajout d'une décision → résumé partagé actualisé. |
| UC-04 - Proposer une adaptation | Artisan | Un élément demandé n'est pas réaliste ou optimal. | Une alternative est discutée puis acceptée ou rejetée. | Créer une proposition → expliquer la contrainte → client répond → proposition validée → brief mis à jour. |
| UC-05 - Confirmer le projet | Client + Artisan | Les éléments essentiels sont | Le projet passe à « confirmé ». | Vérifier résumé → valider éléments clés |

| | | | | |
|---|---|---|---|---|
| | | suffisamment définis. | | → artisan confirme → statut mis à jour. |

# 6. Architecture d'expérience MVP

## 6.1 Écrans essentiels

| Zone | Écran | Rôle |
|---|---|---|
| Public | Page profil artisan | Présenter l'artisan et lancer une demande |
| Public/Auth progressive | Créer une demande | Capturer une intention sans friction excessive |
| Auth client | Mes projets | Voir les projets actifs et leur état |
| Auth partagé | Détail Backprojekt | Résumé + conversation + décisions + statut |
| Auth artisan | Inbox / demandes | Trier et traiter les demandes reçues |
| Auth artisan | Profil / lien public | Configurer la porte d'entrée de ses demandes |

## 6.2 Structure recommandée du détail Backprojekt

- En-tête : nom du projet, client/artisan, date cible, statut.
- Résumé partagé : occasion, portions, budget, style, goûts, contraintes, livraison/retrait, références.
- Éléments ouverts : informations manquantes ou décisions à prendre.
- Fil d'échanges : messages et propositions contextualisés.
- Décisions : éléments validés, avec auteur et date.
- Action principale contextuelle : répondre, demander une précision, accepter, confirmer, clôturer.

# 7. Modèle de domaine minimal

| Entité | Rôle | Champs / relations clés |
|---|---|---|
| User | Identité et authentification | id, name, email, password; peut être client et/ou lié à Artisan |
| Artisan | Profil professionnel / créatif | user_id, display_name, bio, location, public_slug, specialties |
| ProjectRequest | Demande initiale | client_user_id, artisan_id, title, occasion, event_date, servings, budget_min/max, description, status |
| Project | Objet partagé après envoi/acceptation | request_id, client_user_id, artisan_id, status, summary, confirmed_at |
| ProjectAttribute | Données structurées évolutives | project_id, key, value, source, confirmed_at |
| Message | Communication contextualisée | project_id, user_id, body, type, created_at |
| Attachment | Références / images | message_id ou project_id, file_path, |

| | | |
|---|---|---|
| | | mime_type |
| Decision | Décision explicite | project_id, created_by, label, value, status, decided_at |

Simplification possible pour la première itération : fusionner ProjectRequest et Project dans une seule entité Project avec des statuts. Cette option réduit la complexité tant que la distinction métier n'apporte pas encore de valeur réelle.

# 8. Hypothèses et métriques de validation

| Hypothèse | Signal à mesurer | Critère indicatif |
|---|---|---|
| Le formulaire améliore la qualité des demandes | Part des demandes contenant date + portions + budget/style + référence | > 60 % sans relance sur les données de base |
| Le résumé partagé réduit la friction | Nombre de questions répétées / informations redemandées | Tendance à la baisse pendant les tests |
| L'artisan préfère le lien Deliciousthings à un DM libre | Part des prospects redirigés vers son lien public | Usage volontaire récurrent |
| Le Backprojekt devient la source de vérité | Part des projets où résumé/statut sont consultés après un échange | Usage récurrent par les deux côtés |
| Le produit accélère la qualification | Temps perçu avant décision accepter/refuser/clarifier | Meilleur que le processus actuel selon artisan |

## 8.1 North Star provisoire

> **Nombre de projets ayant atteint une compréhension commune mesurable : demande structurée + échange bilatéral + statut accepté/confirmé.**

Cette métrique est plus pertinente pour ce MVP que le simple nombre de comptes, messages ou demandes envoyées.

# 9. Planification d'implémentation

Plan proposé pour un développeur solo disposant déjà d'une base Laravel + Inertia/Vue et de modèles existants. L'objectif est de livrer une version testable rapidement, puis de durcir le produit après observation de vrais échanges.

| Phase | Objectif | Livrables | Durée cible |
|---|---|---|---|
| 0 - Cadrage technique | Stabiliser le modèle et les routes | States, permissions, schéma DB, wireframes simples | 0,5-1 jour |
| 1 - Flux entrée client | Permettre à un prospect d'envoyer une demande | Profil public artisan + formulaire + auth progressive + création Project | 2 jours |
| 2 - Inbox artisan | Recevoir et qualifier | Liste demandes, détail, accepter/refuser/clarifier | 1,5-2 jours |
| 3 - Backprojekt partagé | Créer la source de vérité | Résumé, attributs, statuts, | 2 jours |

| | | | |
|---|---|---|---|
| | | droits d'accès | |
| 4 - Communication | Échanges contextualisés | Messages asynchrones + images + activité | 1,5-2 jours |
| 5 - Décisions & polish | Différencier discussion et accord | Décisions simples, états vides, feedback UI | 1-1,5 jour |
| 6 - Notifications & QA | Rendre le pilote exploitable | E-mails essentiels, tests, sécurité, responsive | 1,5-2 jours |
| 7 - Pilotage terrain | Tester avec utilisateurs réels | 3-5 artisans + demandes réelles + journal de feedback | continu après mise en ligne |

## 9.1 Ordre de développement recommandé

8. Implémenter Project + statuts + Policies avant le chat.
9. Construire le formulaire de demande en premier : c'est la porte d'entrée de la valeur.
10. Construire ensuite l'inbox artisan et les actions de qualification.
11. Créer le détail Backprojekt comme écran central partagé.
12. Ajouter seulement ensuite les messages ; ils doivent enrichir le projet et non devenir le produit.
13. Ajouter les décisions explicites uniquement après que le flux message → mise à jour du brief fonctionne.
14. Brancher les notifications e-mail en dernier, lorsque les événements métier sont stabilisés.

## 9.2 Backlog priorisé

| Priorité | Epic | Contenu |
|---|---|---|
| P0 | Auth & autorisations | Login/register, Policies participant projet, rôles/contexte |
| P0 | Project | Migration, modèle, statuts, création, lecture |
| P0 | Profil public artisan | Slug, CTA « Envoyer une demande » |
| P0 | Formulaire demande | Étapes essentielles + validation + références |
| P0 | Inbox artisan | Liste, filtres statut simples, détail |
| P0 | Qualification | Accepter/refuser/à clarifier |
| P0 | Backprojekt | Résumé partagé, attributs, statut |
| P0 | Messages | Créer/lire messages projet, autorisation |
| P1 | Pièces jointes | Images compressées et contrôlées |
| P1 | Décisions | Créer/valider/rejeter une décision |
| P1 | Notifications | Nouvelle demande, message, changement majeur |
| P1 | UX | États vides, erreurs, loaders, mobile |
| P2 | IA de clarification | Suggestion de questions / synthèse, derrière feature flag |
| P2 | Templates | Demandes par occasion / type de création |

# 10. Notes d’architecture pour le stack actuel

- Laravel : utiliser des Policies sur Project, Message et Decision ; éviter de déduire l’autorisation uniquement depuis le rôle.
- Inertia + Vue : le Backprojekt peut être une page Inertia unique avec composants Summary, OpenItems, Conversation et Decisions.
- Le User type n’a pas besoin d’être figé en « client » ou « artisan » si un même compte peut porter plusieurs contextes. Artisan peut rester un profil spécialisé lié à User.
- Un Client model séparé n’est pas nécessaire pour le MVP si aucune donnée métier propre au client ne le justifie.
- Préférer un enum / state machine simple pour le statut du Project afin d’éviter les transitions incohérentes.
- Les messages doivent être persistés comme événements du projet, mais les informations critiques doivent également exister sous forme structurée dans le Project / ProjectAttribute.
- Ne pas faire de WebSocket au MVP : requêtes Inertia classiques ou polling léger suffisent pour valider la valeur.

# 11. Risques produit et garde-fous

| Risque | Pourquoi il est dangereux | Garde-fou MVP |
|---|---|---|
| Devenir un clone de messagerie | Faible différenciation face à WhatsApp/Instagram | Toujours faire du résumé partagé l’écran principal |
| Formulaire trop long | Le client abandonne avant d’envoyer | Progressif, champs essentiels seulement, possibilité de compléter après |
| Sur-structuration | Le processus paraît bureaucratique | Autoriser texte libre + références, structurer progressivement |
| Double saisie artisan | Il doit encore utiliser ses outils habituels | Limiter le périmètre et tester le gain réel avant CRM avancé |
| IA prématurée | Complexité sans preuve de valeur | MVP 100 % fonctionnel sans IA |
| Marketplace trop tôt | Cold start et dilution du produit SaaS | Flux dirigé vers un artisan précis via son lien public |
| Statuts artificiels | Le workflow ne reflète pas le terrain | Observer 3-5 artisans et ajuster les transitions |

# 12. Plan de validation terrain

## 12.1 Pilote recommandé

- Recruter 3 à 5 pâtissiers / cake designers qui reçoivent déjà des demandes via Instagram ou WhatsApp.
- Leur fournir un lien Deliciousthings à placer temporairement dans leur bio ou à envoyer manuellement à certains prospects.

- Observer 10 à 20 demandes réelles, y compris celles refusées.
- Comparer avec leur ancien processus : qualité de la demande, nombre de relances, temps de qualification, clarté finale.
- Interviewer aussi les clients : le formulaire les aide-t-il à penser leur projet ou leur semble-t-il être une barrière ?

### 12.2 Questions de validation

- À quel moment l'artisan quitte Deliciousthings pour revenir sur WhatsApp ? Pourquoi ?
- Quelles informations manquent encore malgré le formulaire ?
- Quelles informations demandées ne servent presque jamais ?
- Le résumé partagé est-il consulté pendant la discussion ?
- Quelles décisions méritent réellement d'être explicites ?
- Le client comprend-il mieux son propre projet après avoir utilisé le produit ?
- L'artisan serait-il prêt à utiliser son lien Deliciousthings pour une part significative de ses demandes ?

## 13. Définition de "MVP livré"

> **Le MVP est livré lorsqu'un vrai client peut partir d'une idée, envoyer une demande à un vrai artisan, clarifier le besoin avec lui et parvenir à un projet compréhensible par les deux parties sans dépendre d'un autre canal pour reconstruire le contexte.**

- Flux complet utilisable sur mobile.
- Contrôle d'accès correct sur tous les projets/messages.
- Aucun écran essentiel ne dépend de données fictives.
- Les états du projet couvrent le flux réel observé.
- Au moins quelques demandes réelles ont traversé le produit de bout en bout.

## 14. Évolutions après validation

Si la boucle de communication structurée est validée, les extensions doivent renforcer cette compétence plutôt que l'entourer de fonctionnalités génériques.

| Étape | Extension possible | Lien avec la compétence centrale |
|---|---|---|
| V1.1 | Assistant de clarification | Détecte les informations manquantes et propose les bonnes questions |
| V1.2 | Templates par type de création | Accélère la structuration sans rigidifier |
| V1.3 | Estimation / faisabilité | Aide à confronter le désir aux contraintes |
| V1.4 | Portfolio relié aux projets | Montre comment des intentions ont été transformées en réalisations |
| V2 | Découverte / matching | Trouve l'artisan adapté à un projet déjà structuré |
| V2+ | Mémoire collective | Capitalise les patterns de projets et décisions, avec consentement et anonymisation |

# Conclusion

Ce MVP positionne Deliciousthings non comme une marketplace ou une messagerie, mais comme une infrastructure légère de compréhension commune. Le Backprojekt est l'unité centrale : la demande l'initie, la conversation l'enrichit, les décisions le stabilisent et le statut permet aux deux parties de savoir où elles en sont.

> **Principe produit à conserver pendant l'implémentation : chaque fonctionnalité doit soit améliorer la compréhension du projet, soit réduire l'effort nécessaire pour le faire avancer.**